

Java Backend Architecture

Interview Series

Chapter 10.1

Search Fundamentals & Inverted Index

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 10.1 – Search Fundamentals & Inverted Index

Introduction

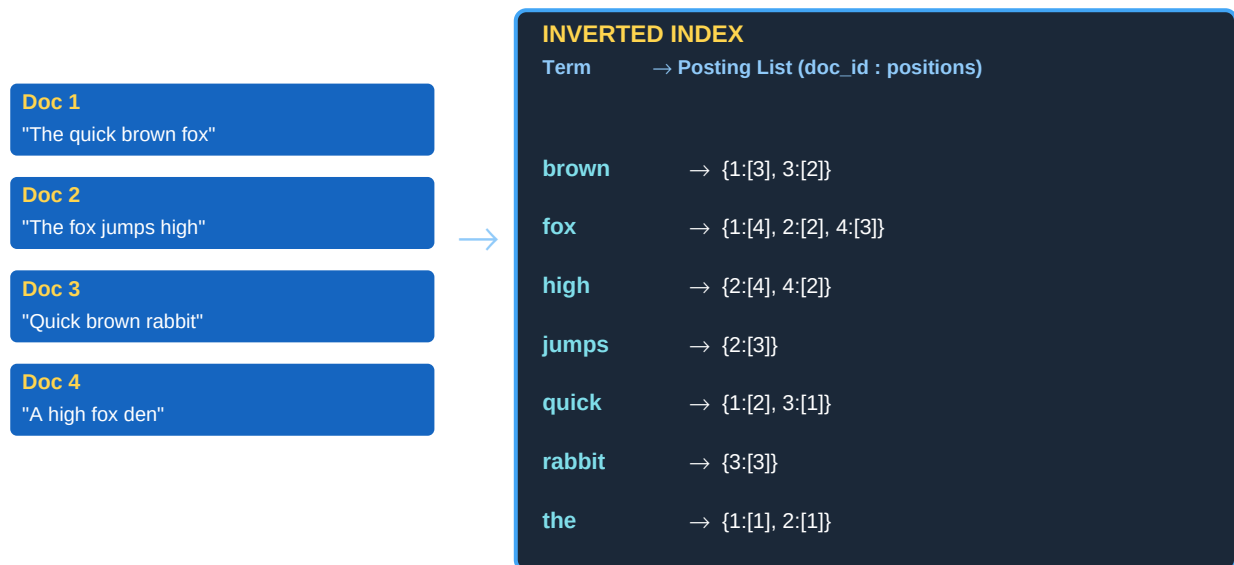
Full-text search engines like Elasticsearch, OpenSearch, and Apache Solr are built on a core data structure called the **inverted index** — a mapping from terms (words) to the documents that contain them, along with positional information. Unlike a relational database's B-tree index (which maps rows to values), an inverted index enables sub-second ranked full-text search across billions of documents. Understanding how text is analyzed, indexed, and scored is fundamental to building effective search experiences in Java backend systems.

Core Concepts & Definitions

Inverted Index	Core data structure: maps each unique term to a posting list of documents (+ positions) containing it.
Term Dictionary	Sorted list of all unique terms in the index; enables $O(\log N)$ term lookup.
Posting List	Per-term list of (doc_id, term_frequency, positions) entries; stored in compressed form.
Document	Unit of search: a JSON object indexed into Elasticsearch/Lucene.
Field	Named attribute of a document; each field gets its own inverted index.
Token / Term	A single indexable unit produced by text analysis (e.g., "quick", "fox").
Analyzer	Pipeline: Character Filter → Tokenizer → Token Filters; converts raw text to tokens.
Tokenizer	Splits text into tokens (standard, whitespace, edge_ngram, etc.).
Token Filter	Transforms token stream: lowercase, stop words, stemmer, synonyms, phonetic.
TF (Term Frequency)	Number of times a term appears in a document; higher = more relevant (with diminishing returns).
IDF (Inverse Document Frequency)	Rarity of a term across all documents; rare terms are more discriminating.
BM25	Okapi BM25: probabilistic ranking function used by Elasticsearch; improved TF-IDF.
Segment	Immutable sub-index in Lucene; multiple segments merged periodically.
Shard	An Elasticsearch unit of distribution; a Lucene index.
Forward Index	Maps document → terms (used for reconstruction); complement of inverted index.
doc_values	Column-oriented field storage used for sorting, aggregations, and scripting.

Inverted Index Structure

Inverted Index Structure



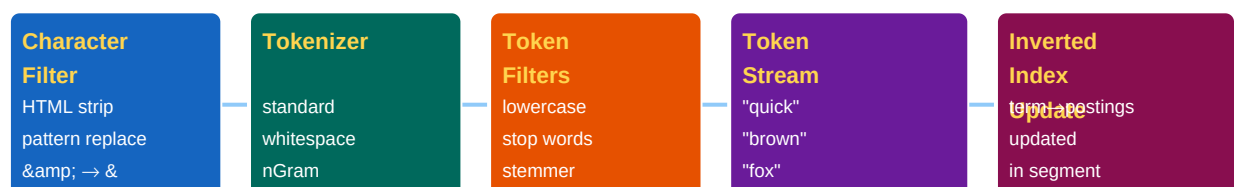
Term Dictionary: sorted unique terms. Posting List: {doc_id: [positions]}

Lookup "fox" in $O(\log N)$ via binary search on term dictionary → return docs 1, 2, 4

The term dictionary stores all unique tokens in sorted order, enabling binary search in $O(\log N)$. Each term points to a posting list that stores compressed doc IDs and positions using delta encoding and variable-byte compression. Phrase queries ("quick fox") require position information; simple term queries only need doc IDs.

Text Analysis Pipeline

Input: "The Quick Brown Fox! Jumps Over"



Query time: same pipeline applied to search terms before matching against index

The analysis pipeline is applied at both index time and query time. The same analyzer must be used for both phases — if you index with a stemmer but query without one, "running" won't match "run". Custom analyzers are configured in index settings and referenced in field mappings.

BM25 Relevance Scoring

BM25 Relevance Scoring (Elasticsearch Default)

BM25(D,Q) = sum over terms t in Q of:

$$\text{IDF}(t) * [\text{TF}(t,D) * (k1 + 1)] / [\text{TF}(t,D) + k1 * (1 - b + b * |D|/\text{avgDL})]$$

where k1=1.2 (TF saturation), b=0.75 (length normalization)

IDF(t)

$\log(1 + (N - df + 0.5)/(df + 0.5))$
N=total docs, df=docs with term

TF(t,D)

frequency of term t in doc D
Saturation: diminishing returns

Length Norm

b=0.75 penalizes long docs
b=0 disables normalization

BM25 replaced TF-IDF as Elasticsearch default in version 5.0. Tune k1/b per index.

How Lucene Stores the Inverted Index

Lucene (the library underlying Elasticsearch) organizes data into **segments** — immutable sub-indexes. Each segment contains: the term dictionary (.tim/.tip files), posting lists (.doc, .pos, .pay), stored fields (.fdt), doc_values (.dvd), and term vectors (.tvd). Writes create new segments; deletes mark documents with a tombstone. Segment merges (background) consolidate small segments and remove deleted docs. This immutable design enables efficient disk I/O and OS page caching.

```
# Lucene segment files overview: # .tim / .tip – Term Index/Dictionary (FST-compressed term
lookup) # .doc – Posting lists (document frequencies + skip data) # .pos – Position information
for phrase queries # .pay – Payloads and offsets # .fdt / .fdx – Stored field values (for
_source retrieval) # .dvd / .dvm – doc_values (columnar; for sort/agg/script) # .tvd / .tvx –
Term vectors (per-doc term stats; for highlighting) # .nvm / .nvd – Norms (field length
normalization factors) # .liv – Live documents (tombstone bitmap for deletes) # segments_N –
Segment commit point (list of active segments) # Segment merge policy (TieredMergePolicy
default): # index.merge.policy.max_merge_at_once: 10 # index.merge.policy.segments_per_tier:
10 # index.merge.policy.floor_segment: 2mb
```

Building an Analyzer in Elasticsearch

```
PUT /products { "settings": { "analysis": { "char_filter": { "html_strip_filter": { "type":
"html_strip" } }, "tokenizer": { "my_ngram": { "type": "edge_ngram", "min_gram": 2, "max_gram":
10, "token_chars": ["letter", "digit"] } }, "filter": { "my_stop": { "type": "stop",
"stopwords": "_english_" }, "my_stem": { "type": "stemmer", "language": "english" }, "my_syn":
{ "type": "synonym", "synonyms": ["quick,fast,rapid", "fox,canine"] } }, "analyzer": {
"product_analyzer": { "type": "custom", "char_filter": ["html_strip_filter"], "tokenizer":
"standard", "filter": ["lowercase", "my_stop", "my_stem", "my_syn"] },
"autocomplete_analyzer": { "type": "custom", "tokenizer": "my_ngram", "filter": ["lowercase"]
} } } }
```

Testing an Analyzer

```
# Test analyzer output via _analyze API POST /products/_analyze { "analyzer":
"product_analyzer", "text": "The Quick Brown Fox!\n Jumps over <b>lazy</b> dogs" } # Response: #
tokens: ["quick", "brown", "fox", "jump", "lazi", "dog"] # Note: HTML stripped, lowercased,
stop words removed, # stemmed (jumps→jump, lazy→lazi, dogs→dog) # Analyze a specific field
POST /products/_analyze { "field": "name", "text": "Running shoes for athletes" }
```

TF-IDF vs BM25 Comparison

Property	TF-IDF (classic)	BM25 (ES default)
TF Saturation	Linear (unlimited)	Logarithmic (k1 parameter caps growth)
Length Normalization	Cosine normalization	b parameter (0=none, 1=full)
Doc frequency weighting	$\log(N/df)$	$\log(1 + (N-df+0.5)/(df+0.5))$
Tuning	No parameters	k1 (1.2), b (0.75) per-field
Performance on short queries	Adequate	Better (IDF smoothing)
Elasticsearch version	Pre-5.0	5.0+ (current default)

Interview Questions & Answers

Q1. What is an inverted index?

An inverted index maps each unique term to a posting list of documents containing that term, along with term frequency and position information. It is the inverse of a forward index (document → terms). It enables $O(1)$ or $O(\log N)$ term lookups, making full-text search over billions of documents feasible.

Q2. How is an inverted index different from a B-tree index?

B-tree index: maps column values → row pointers; efficient for equality/range queries on structured data. Inverted index: maps text terms → document lists; efficient for full-text search, phrase queries, boolean queries, and relevance ranking. B-tree lookups are exact; inverted index lookups involve scoring.

Q3. What is a posting list?

A per-term sorted list of (doc_id, term_frequency, positions) entries. Stored in compressed format: doc IDs as delta-encoded integers (gap compression), frequencies as variable-byte integers, positions as delta-encoded. Skiplists enable fast seeking within large posting lists.

Q4. What is a Lucene segment?

An immutable mini-index within a Lucene shard. Contains its own term dictionary, posting lists, stored fields, and doc_values. Writes create new segments; reads must search all segments and merge results. Background merges consolidate segments and purge deleted docs (tombstones).

Q5. Why are Lucene segments immutable?

Immutability simplifies caching (OS page cache), concurrent reads without locks, crash recovery (segments are complete; partial writes don't corrupt), and compression efficiency. Updates are implemented as delete + re-index; deletes are lazy tombstones until merge.

Q6. What is text analysis in Elasticsearch?

Text analysis converts raw text to tokens for indexing and querying. Pipeline: Character Filters (HTML strip, pattern replace) → Tokenizer (splits text; standard, whitespace, edge_ngram) → Token Filters (lowercase, stop words, stemmer, synonyms, phonetic). Same pipeline applied at index and query time.

Q7. What is the difference between a tokenizer and a token filter?

Tokenizer: splits the character stream into tokens; exactly one tokenizer per analyzer. Token filter: transforms, adds, or removes tokens from the stream; multiple allowed per analyzer. Example: whitespace tokenizer splits on spaces; lowercase filter converts each token to lowercase.

Q8. What is the standard analyzer?

Elasticsearch's default analyzer: Unicode Text Segmentation tokenizer (splits on whitespace/punctuation), lowercase filter, stop word filter (disabled by default). Suitable for most Latin-language text. For languages with no word boundaries (Chinese, Japanese) use the dedicated language analyzers.

Q9. What is BM25 and why does Elasticsearch use it?

BM25 (Best Match 25) is a probabilistic ranking function that improves on TF-IDF with TF saturation (diminishing returns via k_1) and configurable length normalization (b). Elasticsearch adopted it in 5.0 because it produces better rankings for short queries and real-world text distributions.

Q10. What is IDF and why does it matter?

Inverse Document Frequency: measures how rare a term is across all documents. $IDF = \log((N - df + 0.5) / (df + 0.5) + 1)$. Rare terms get high IDF weight; common terms (like 'the') get low weight. This ensures discriminating terms contribute more to ranking than ubiquitous ones.

Q11. What is TF saturation in BM25?

Traditional TF-IDF scores increase linearly with term frequency. BM25 applies saturation: $score(TF) = TF * (k_1 + 1) / (TF + k_1)$. As TF grows, the score asymptotically approaches $(k_1 + 1)$. $k_1 = 1.2$ (default): a term appearing 100 times scores only slightly more than one appearing 5 times. Prevents documents from gaming scores by repeating keywords excessively.

Q12. What is the b parameter in BM25?

b controls length normalization. $b=0$: length not considered. $b=1$: full normalization (score scales with length). Default $b=0.75$ partially normalizes: longer documents are slightly penalized. A 1000-word doc with 5 occurrences of a term scores lower than a 100-word doc with 5 occurrences.

Q13. What are doc_values and why is it separate from the inverted index?

`doc_values` store field values in a columnar (document-oriented) format on disk. Used for sorting, aggregations, and scripting — operations that need to access field values for many documents. Inverted index is term-oriented (term $\rightarrow$ docs). `doc_values` are document-oriented (doc $\rightarrow$ values). Both are stored; toggling `fielddata=true` loads text field values into heap instead (deprecated).

Q14. What is fielddata and why is it problematic?

`fielddata` loads text field values into JVM heap for aggregations/sorting. Expensive: uninverts the inverted index at query time, consumes significant heap. For keyword fields, `doc_values` are used instead (off-heap, efficient). `fielddata` is deprecated for text fields; use keyword sub-fields.

Q15. What are stored fields in Elasticsearch?

`_source`: complete original JSON document stored verbatim; returned in search results. Store individual fields: 'store: true' in mapping stores that field separately. Stored fields are compressed row-by-row in `.fdt` files. Disabling `_source` saves disk but prevents updates and reindex; not recommended for production.

Q16. What is a term vector?

Per-document record of all terms, their frequencies, and positions. Enabled per field ('term_vector: with_positions_offsets'). Used for: highlighted snippets (`fast_vector_highlighter`), more-like-this queries, and custom relevance debugging. Adds disk overhead; enable only when needed.

Q17. What is the difference between 'text' and 'keyword' field types?

text: full-text analyzed field; inverted index for full-text search; cannot sort or aggregate directly. keyword: not analyzed; stored as-is; supports exact match, sort, aggregation, and term-level queries. Common pattern: map name as text for search + sub-field name.keyword as keyword for sorting/faceting.

Q18. What is an n-gram tokenizer?

Generates all substrings of length n from a token. Example: 'fox' with $n=2 \rightarrow$ ['fo', 'ox']. edge_ngram: only prefixes ('f', 'fo', 'fox'). Used for autocomplete: index 'elasticsearch' as edge n-grams; query 'elasti' matches. Trade-off: large index size.

Q19. What is stemming?

Reducing a word to its root form. English stemmer: 'running' $\rightarrow$ 'run', 'jumps' $\rightarrow$ 'jump', 'dogs' $\rightarrow$ 'dog'. Both index and query terms are stemmed, so 'running shoes' matches 'run shoe'. Lucene's SnowballPorterFilter. Trade-off: over-stemming (unrelated words share stem) and under-stemming (related words differ).

Q20. What is lemmatization and how does it differ from stemming?

Lemmatization uses linguistic rules to reduce words to their dictionary form (lemma): 'better' $\rightarrow$ 'good', 'ran' $\rightarrow$ 'run'. More accurate than stemming but slower. Elasticsearch doesn't include a lemmatizer natively; use the analysis-icu plugin or external NLP. Stemming is algorithmic and fast; lemmatization is dictionary-based and precise.

Q21. What are synonyms in text analysis?

Synonym token filter expands or replaces tokens during analysis. Index-time synonyms: index all synonyms; faster query. Query-time synonyms: apply at query; more flexible (can update synonyms without reindex). Format: 'quick,fast,rapid' (equivalents) or 'quick => fast' (directional). Stored in synonym file or inline in mapping.

Q22. What is the purpose of stop words?

Stop words (the, a, is, are) are filtered out during analysis because they appear in almost every document and contribute little discriminating power. Removing them reduces index size and speeds up queries. Risk: phrase queries may fail if stop words are in the phrase ('to be or not to be'). Use shingles (bigrams) to preserve phrase structure without storing stop words.

Q23. What is a Boolean query model?

Documents must satisfy boolean conditions: MUST (AND), SHOULD (OR), MUST_NOT (NOT). Elasticsearch's bool query: must clauses required + contribute to score, filter clauses required but don't score, should clauses optional but boost score, must_not excludes. Filter clauses are cached; must clauses are not.

Q24. What is relevance and how is it computed?

Relevance score = measure of how well a document matches the query. Computed by BM25 per field. Total score: sum of BM25 scores across matched fields (with optional per-field boost). Tie-breaking, function_score, or learning-to-rank can modify the base BM25 score. Explain API: GET /index/_explain/{id} to see score decomposition.

Q25. What is the Lucene FST (Finite State Transducer)?

FST is a compact data structure used for the term dictionary. It maps byte sequences (terms) to output values (file offsets into posting lists) with minimal memory. Shares common prefixes and suffixes. Loaded into memory on open for sub-microsecond term lookups. Core to Elasticsearch's speed — the entire term dictionary fits in memory via FST.

Q26. What is the difference between index-time and query-time analysis?

Index-time analysis: applied when documents are indexed; determines what tokens are stored. Query-time analysis: applied to search terms before matching. Both must be compatible — ideally the same analyzer. Mismatch: indexing with stemmer but querying without $\rightarrow$ 'running' in query doesn't match 'run' in index.

Q27. What is a percolator in Elasticsearch?

Inverted search: instead of indexing documents and querying, you index queries and percolate (match) documents against them. Use case: real-time alerting — store user alert queries; when a new article arrives, percolate to find which users' alerts match. Implemented via percolate query.

Q28. What are shingles and when are they used?

Shingles (shingle filter) generate word bigrams/trigrams: 'quick brown fox' → 'quick', 'quick brown', 'brown fox'. Used to approximate phrase matching without expensive positional lookups, and to preserve context for stop words. Common in phrase_suggest and in high-performance phrase-query alternatives.

Q29. How does Elasticsearch handle near real-time (NRT) search?

Lucene writes data to an in-memory buffer. refresh (default every 1s) creates a new segment from the buffer and makes it searchable — without an fsync. flush/commit persists segments to disk with an fsync for durability. NRT = searchable within ~1 second, not immediately. index.refresh_interval: -1 disables refresh for bulk indexing.

Q30. What is the translog in Elasticsearch?

The translog (transaction log) records every indexing operation before the Lucene commit. On crash before commit, the translog is replayed to recover unwritten operations. index.translog.durability: request = fsync on every request (default), async = fsync every sync_interval. Async improves throughput at the cost of up to sync_interval data loss.

Q31. How does Elasticsearch handle document updates?

Elasticsearch is built on an immutable Lucene index. An update: 1) reads the existing document, 2) applies the partial update, 3) reindexes the full document as a new version, 4) marks the old version as deleted (tombstone). The delete is purged during segment merge. Partial updates via _update API reduce network transfer but still do a full re-index internally.

Q32. What is the _routing parameter?

By default, Elasticsearch routes documents to shards by: $\text{hash}(\text{doc_id}) \% \text{num_shards}$. Custom routing: PUT /products/_doc/42?routing=category-5 sends the doc to a specific shard group. Advantage: related docs co-located; queries with routing=category-5 hit only relevant shards. Risk: hot shards if routing key has skewed distribution.

Q33. Describe the lifecycle of an indexed document.

1) Client sends PUT/POST to any node (coordinating). 2) Node determines target shard via routing. 3) Request forwarded to primary shard. 4) Primary writes to translog + Lucene buffer. 5) Request forwarded to replica shards. 6) Response sent to client (write_consistency met). 7) Refresh (1s later): buffer flushed to new segment → searchable. 8) Flush (every 30min or translog size): Lucene commit + translog truncation.